AUTOGRADER: A RULE-DRIVEN EVALUATOR FOR C PROGRAMS

A Project Report

Submitted in Partial Fulfilment
of the Requirements for the Course Assignment

TEAM EVEREST

Kanishk Mishra | 2611CS20

Roshan Jha | 2611CS23

Mantosh Ravi | 2611CS09

Shivanshu Raj | 2611CS26

Under the Guidance of
Dr. Arijit Mondal



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY PATNA

October 2026

ABSTRACT

Assessing C programming assignments requires more than checking sample output. A program may compile successfully and still fail at numeric boundaries, contain unsafe memory operations, or be difficult to understand. AutoGrader integrates these concerns into a rule-driven workflow that combines deterministic software checks with bounded local AI review.

The implemented system accepts a single C source file and an instructor-defined JSON configuration. A Python coordinator validates the rules, performs compilation and smoke checks, dispatches nine independent metric analyzers, and calculates one weighted grade. Correctness tests, a trusted arithmetic oracle, compiler diagnostics, sanitizer executions and local source similarity provide tool-based evidence. Four Ollama specialists evaluate code structure, modularity, complexity and readability using explicit rubrics, temperature zero, a fixed seed and validated JSON responses.

A React frontend connects to a FastAPI backend to support source upload and editing, rubric configuration, asynchronous evaluation status, feedback, saved history and JSON/text downloads. Direct file handling is sufficient for the current workload; retrieval-augmented generation and a vector database are not required. The implementation separates interface code, orchestration, analysis, model communication and execution control so the modules can be maintained independently.

Reproducibility is addressed through configuration validation, recorded runtime identities and content-based verdict caching. Cached replay preserves a completed grade for an unchanged evaluation key, while fresh inference and timing outcomes can still vary. Verification on the Windows creation host recorded 34 passing software-level tests and nine skipped Ubuntu runtime/model tests. React production building and selected browser/API interactions were also checked. This report distinguishes these verified results from real Docker/Ollama grading, which remains a target-host acceptance task.

Keywords: AutoGrader; Agentic AI; C programming; automated assessment; Ollama; React; FastAPI; sandboxing; rule-driven grading; reproducibility.

Report status

This document describes the implemented codebase and recorded progress. It does not present the assignment's illustrative scores as measured project results. Team responsibilities and the four-week schedule are proposed allocations for collaboration, not unverified claims about individual past contributions.

TABLE OF CONTENTS

ABSTRACT . . . . . **i**

1 Introduction **1**

1.1 Motivation 1

1.2 Problem statement 1

1.3 Objectives 1

1.4 Scope and report organization 1

2 System Requirements **2**

2.1 Functional requirements 2

2.2 Non-functional requirements 2

2.3 Evaluation metrics and input/output limits 3

2.4 Software requirements 3

3 System Architecture **4**

3.1 Overall architecture 4

3.2 Coordinator and rule configuration 4

3.3 Compilation and execution module 5

3.4 Evaluation workflow and failure handling 5

3.5 Queueing and report generation 5

4 Rule-Driven Evaluation and Analyzers **6**

4.1 External rules and weighted grading 6

4.2 Functional, compiler and edge-case methods 6

4.3 Memory-safety and originality methods 6

4.4 Local LLM specialist agents 7

4.5 Prompt and inference configuration . . . 7
4.6 Reproducibility policy . . . 7

5 Implementation Methodology . . . 8

5.1 Modular development approach . . . 8
5.2 Input validation and file handling . . . 8
5.3 Code integration examples . . . 9
5.4 Compiler and sanitizer integration . . . 9
5.5 Failure policy in code . . . 9
5.6 React frontend and FastAPI backend . . . 10
5.7 Reports and interface boundaries . . . 10
5.8 Team allocation and four-week plan . . . 11

6 Testing and Results . . . 12

6.1 Testing approach and verified results . . . 12
6.2 Pending target-environment tests . . . 12
6.3 Assignment sample and defined arithmetic behavior . . . 13
6.4 Illustrative scores versus measured results . . . 13
6.5 Reproducibility acceptance procedure . . . 13

7 Usefulness and Limitations . . . 14

7.1 Practical uses . . . 14
7.2 Conditions for useful feedback . . . 14
7.3 Concrete unreliable-verdict cases . . . 15
7.4 Security and human-review boundary . . . 15

8 Conclusion and Future Work . . . 16

8.1 Conclusion . . . 16

8.2 Future work 16

8.3 Final submission checklist 16

Appendix A Example Rule Configuration 17

A.1 Rule-editing guidance 17

Appendix B Feedback and JSON Report Contract 18

B.1 Failure evidence 18

B.2 Report storage and transport 18

Appendix C Implementation Evidence 19

C.1 Codebase, environment and recorded checks 19

C.2 Ubuntu setup and acceptance commands 19

C.3 Actual React interface preview 20

REFERENCES 21

CHAPTER 1
INTRODUCTION

1.1 Motivation

Programming assignments test a student's ability to turn a problem statement into an executable solution. In a C laboratory, instructors must compile submissions, choose representative inputs, inspect errors and apply the marking scheme consistently. Repeating these operations across a class takes time and makes detailed feedback difficult to provide.

Output-only grading is incomplete. For example, adding two ordinary integers may work while adding values near the limits of `int` triggers undefined behavior. Similarly, a program can return the expected result on the tested path while retaining a memory defect elsewhere. A useful evaluator should explain both the observed behavior and the limits of the evidence.

1.2 Problem statement

The project implements an evaluator that receives a C program and external rules, performs controlled checks with minimal intervention, and returns an explainable grade. It must survive compilation errors, crashes and nontermination, evaluate each metric separately, preserve specific failure examples and make repeated verdicts auditable. These goals are derived from the supplied assignment [1].

1.3 Objectives

1. Separate instructor rules from application source code and reject invalid configurations.

2. Collect bounded compilation and execution evidence before requesting semantic judgments.

3. Combine nine analyzer results using explicit, validated weights and a documented failure policy.

4. Provide an accessible local interface and downloadable reports with counterexamples.

5. Record evaluation identities, distinguish unknown results from zero scores, and support cached replay.

1.4 Scope and report organization

The current product evaluates one UTF-8 C source file through a local instructor workspace. Ubuntu is the execution target; React and FastAPI provide the interface and transport. Chapters 2-4 cover requirements, architecture and grading methods. Chapters 5-6 document implementation and verification. Chapters 7-8 discuss applications, limitations and future work. The appendices provide a valid reduced rule example, report contracts, and implementation evidence.

CHAPTER 2
SYSTEM REQUIREMENTS

2.1 Functional requirements

ID	Requirement	Implemented response
FR1	Input and rule management	Load a single .c file and JSON rules; validate fields, metric names, weights and file limits.
FR2	Compilation and restricted execution	Compile inside Docker; enforce wall-time, memory, process and output limits.
FR3	Independent analysis	Dispatch each selected metric through a registered analyzer with a shared result contract.
FR4	Grading and feedback	Calculate weighted contributions; retain inputs, expected output, actual output and diagnostics.
FR5	Originality warnings	Compare with a bounded local corpus and flag threshold matches for human review.
FR6	Interface and reports	Expose React uploads/editing, rules, real job state, history, and JSON/TXT downloads.

Table 2.1. Functional requirements and their corresponding implementation responsibilities.

2.2 Non-functional requirements

Reproducibility and reliability. Scores must be traceable to rules and evidence. Failed tools must be distinguishable from a student program failing a valid test. Completed evaluations are replayed using an input/runtime fingerprint; independent recomputation requires additional control and validation.

Security and maintainability. Student code is treated as untrusted. The execution boundary constrains filesystem, network and resource access. Explicit interfaces allow analyzers to be changed without embedding grading policy in the frontend.

Usability and extensibility. Instructors can inspect rules and feedback, while machine-readable reports support further processing. A new subjective metric can use the generic rubric analyzer. A new tool capability requires a reviewed analyzer implementation and registry entry.

2.3 Evaluation metrics and input/output limits

Metric	Primary purpose	Weight
Correctness	Match instructor test outputs	20%
Syntax	Strict compilation and diagnostics	10%
Structure	Function length, decisions and nesting	10%
Modularity	Meaningful function decomposition	10%
Logical / edge cases	Boundary and generated-input behavior	10%
Complexity	Source-based efficiency estimate	10%
Memory safety	Instrumented runtime diagnostics	10%
Readability	Clarity, naming and layout	10%
Originality	Similarity triage against local references	10%

Inputs are limited to one 12 KB source file, up to 100 instructor tests, 50 additional random cases, 20 configured metrics and 20 reference files. The standard arithmetic generator also includes fixed boundary cases. Web rules use inline tests; the CLI can load matching `.in/.out` pairs below the rules directory.

2.4 Software requirements

Component	Purpose / target
Python 3.10+	Coordinator, API, file operations and report generation.
React + Vite / Node.js 22.12+	Interactive frontend and production asset build.
FastAPI + Uvicorn	JSON endpoints, bounded evaluation queue and static frontend serving.
Ollama	Local model runner; example model: qwen2.5-coder:7b.
Docker / Ubuntu 24.04	Target environment for the restricted C compiler/runtime image.
GCC / ASan / UBSan	C11 builds and selected memory/undefined-behavior checks.

Python dependencies are documented in `requirements.txt`; frontend dependencies are locked in `web/package-lock.json`. Cppcheck and Lizard are not used in the current implementation. Structure and complexity are reviewed by local Ollama specialists, not presented as results from tools that were never installed.

CHAPTER 3
SYSTEM ARCHITECTURE

3.1 Overall architecture

The interface, transport, grading service and analyzers are separate layers. The coordinator owns execution decisions; LLM output cannot introduce tool calls or override test results. All specialists return score, status, feedback and evidence under one validated contract.

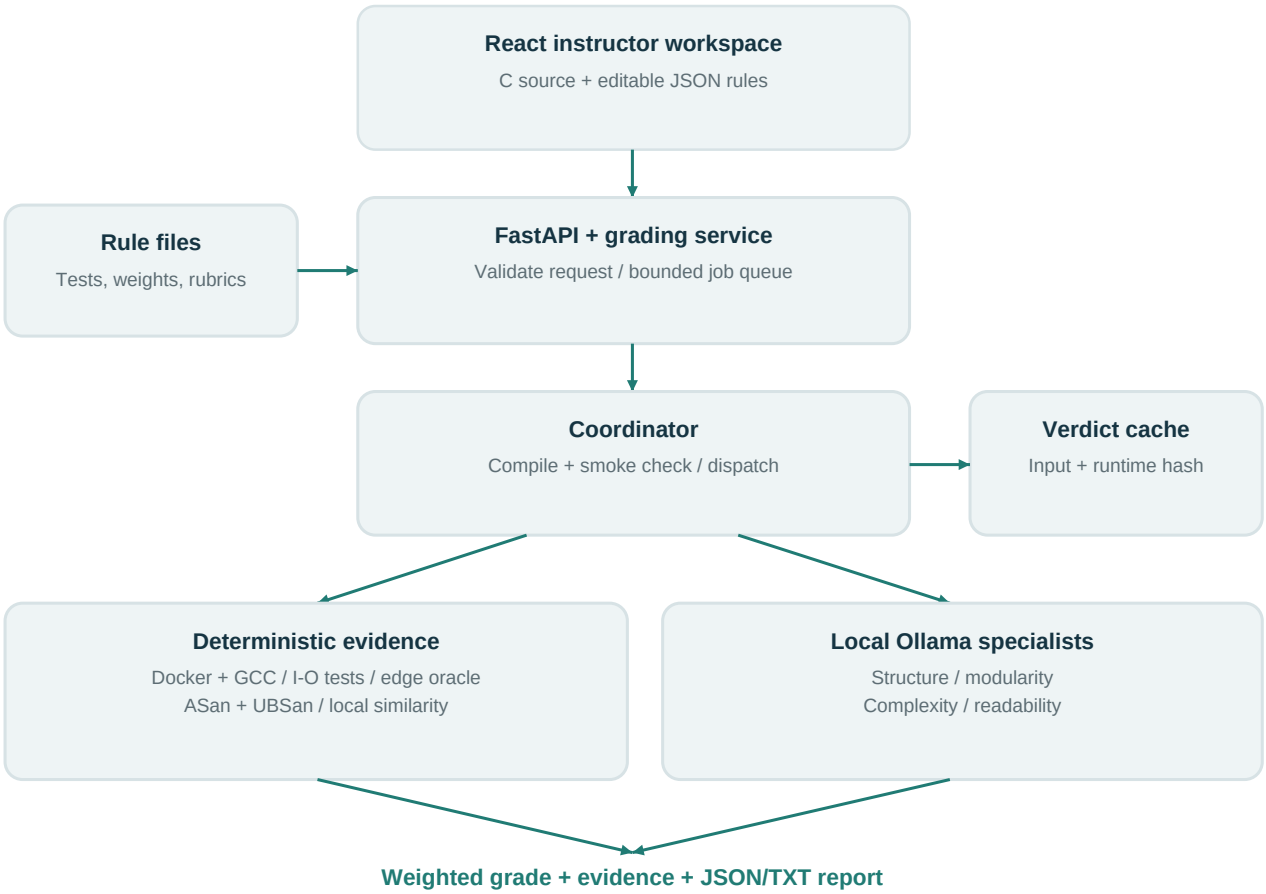


Figure 3.1. Implemented architecture: React and FastAPI connect the instructor to a coordinator, deterministic evidence tools, four local LLM specialists and a verdict cache.

3.2 Coordinator and rule configuration

The coordinator validates analyzer identifiers, resolves the model and Docker image identity, computes the evaluation fingerprint and checks the cache. Rules specify the problem contract, test data, metric weights and rubric parameters. Since these values live outside the code, an instructor can change the marking policy without rewriting the dispatch logic.

The cache key covers the source, expanded rules/tests, corpus, prompt, grader implementation and recorded runtime identities. Cached results are returned with a replay indicator. Failed service evaluations are not cached, so the next request can retry after the problem is corrected.

3.3 Compilation and execution module

A fresh container handles each compilation or execution request. Source and binary data enter through standard input; the host source directory and Docker socket are not mounted. The container has no network access, a read-only root and a bounded temporary filesystem. The supervisor retains only the capabilities needed to drop child privileges and terminate child processes.

Compiler and submitted-program children run as UID/GID 65534. Their output is bounded; timeouts terminate the process group. Normal runs use both container physical-memory limits and a virtual-address limit. Sanitizer runs have a separate physical-memory budget because AddressSanitizer reserves a large virtual address range [4].

3.4 Evaluation workflow and failure handling

Stage	Normal action	Failure behavior
Validation	Check source, rules and requested analyzers.	Reject invalid requests before grading starts.
Compile	Build using fixed GCC options.	Record diagnostics; execution metrics receive skipped/zero-credit status.
Smoke run	Run the first test to assess process health.	Flag timeout/crash; skip execution-based metrics while source checks may continue.
Analyzer dispatch	Collect metric scores and evidence.	Preserve other findings; tool/model errors make the final grade unavailable.
Aggregation	Apply the original metric weights.	Do not normalize missing evidence into an apparently complete grade.
Persistence	Save JSON/text and cache valid completed verdicts.	Surface storage/service failures rather than claim a successful report.

Table 3.1. The implementation distinguishes a failing submission from a failing evaluation service.

3.5 Queueing and report generation

The API accepts one running evaluation and one waiting request. A background worker updates real job status, which React polls. Saved reports persist on disk; recent job states are bounded in memory. JSON and text reports are generated from the same underlying result so their scores and feedback remain consistent.

CHAPTER 4
RULE-DRIVEN EVALUATION AND ANALYZERS

4.1 External rules and weighted grading

Each metric has a name, analyzer identifier, weight and parameters. Rules are validated with Pydantic. The normalized score s_i and weight w_i must lie within [0, 1], and the weights must sum to 1. Invalid totals are rejected rather than silently rescaled.

$$G = 100 \times \text{sum}(w_i \times s_i), \quad \text{sum}(w_i) = 1.$$

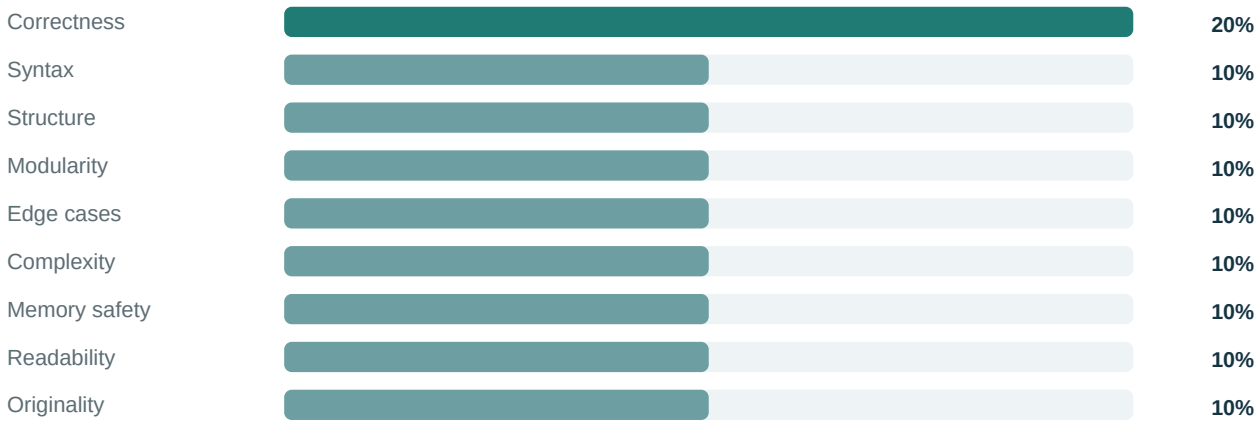


Figure 4.1. Example rubric distribution: correctness has 20% weight and eight other metrics have 10% each. The chart describes configured maximum contributions, not measured student scores.

4.2 Functional, compiler and edge-case methods

Correctness is the fraction of instructor test cases that pass the selected exact-output or token comparison. Each failing case retains the actual input and expected output. Syntax receives credit for a strict C11 build with warning flags and `-Werror`. Compilation flags are fixed in the sandbox; rules cannot inject arbitrary command-line options.

The built-in addition oracle uses Python integers to calculate mathematical sums. It includes zero, negative values, cancellation and both int32 extremes, followed by seeded random cases. For another problem, an instructor must supply tests and an appropriate trusted oracle; model-generated answers are not treated as ground truth.

4.3 Memory-safety and originality methods

ASan/UBSan checks execute an instrumented build on selected cases and record runtime diagnostics. A clean run only covers exercised paths. Originality uses token-sequence similarity against local references. The default policy is advisory: a high match raises a flag without an automatic misconduct penalty. No online search or reliable AI-authorship detection is claimed.

4.4 Local LLM specialist agents

Specialist	Rubric focus	Important boundary
Structure	Estimated decision complexity, function length and nesting.	The result is semantic model judgment, not a formal static-analysis proof.
Modularity	Meaningful helpers and responsibility separation.	Decomposition expectations must fit the task; more functions are not always better.
Complexity	Time/auxiliary-space reasoning against the required bound.	No timing-curve confirmation or universal asymptotic proof is claimed.
Readability	Names, layout, comments and clear input/output flow.	Deduct only against instructor anchors and cite actionable source findings.

4.5 Prompt and inference configuration

Each specialist uses an independent conversation containing its role, problem specification, rubric, numbered source and compiler evidence. The shared system prompt instructs the model to treat student comments and strings as untrusted evidence, avoid inventing test results and state uncertainty. Responses use Ollama's structured JSON interface and are validated before acceptance [3].

Parameter	Current configuration
Model	qwen2.5-coder:7b
Temperature / seed	0 / 42
Context / output setting	16,384 / 2,048
Validation	Finite score in [0,1], structured feedback, and real source-line references.
Input budget	Oversized reviews are rejected rather than silently truncating submitted code.

4.6 Reproducibility policy

The practical guarantee is persisted verdict replay for the same evaluation key. Fixed settings reduce variation, but temperature zero alone does not ensure identical fresh generations on all hardware or model versions. Runtime thresholds can also vary under load. A cohort should retain its code, rules, model/image identities and verdict cache; a separate recomputation study should compare fresh results before stronger determinism claims are made.

CHAPTER 5
IMPLEMENTATION METHODOLOGY

5.1 Modular development approach

The application combines traditional software control with bounded agent review. Python owns validation, file operations, execution and arithmetic. Local LLM specialists interpret qualitative rubrics. React has no grading logic: it sends requests and displays backend results. The explicit boundaries make module ownership and testing manageable for a four-member team.

Code module	Responsibility
app.py	Entry point for web, API, CLI grading, doctor checks and optional legacy Gradio.
api.py / backend.py	HTTP validation, job lifecycle, health/history endpoints and service invocation.
web/src/main.jsx	React editor, upload, rule configuration, status, feedback and JSON downloads.
grader/models.py / files.py	Shared contracts, bounded file loading, path checks and atomic persistence.
grader/coordinator.py	Sanity sweep, explicit registry dispatch, weighting and verdict caching.
grader/ollama_client.py	Loopback-only model calls, timeouts, schema validation and identity discovery.
grader/sandbox.py / sandbox/worker.py	Restricted Docker operations and bounded compiler/runtime processes.
grader/analyzers/	Nine independent analyzer modules plus the reusable rubric base.
grader/reporting.py / locking.py	Consistent JSON/text reports and serialized grading/cache writes.

Table 5.1. Implemented module map. Detailed setup commands and commented dependencies accompany the source package.

5.2 Input validation and file handling

The backend validates rules before execution. Browser requests include source text, a safe filename and the rules object; filesystem traversal and oversized payloads are rejected. The local comparison corpus is bounded. Direct file processing is sufficient, so the project avoids an unnecessary retrieval index, embedding model and vector database.

5.3 Code integration examples

The following excerpts are taken from the implemented project. They demonstrate that the coordinator calculates grades from normalized results and explicitly withholds an official grade when evidence is incomplete.

```
# grader/coordinator.py
def aggregate(metrics):
    incomplete = any(m["status"] == "error" for m in metrics)
    points = sum(m["contribution"] for m in metrics)
    return None if incomplete else round(points, 4), round(points, 4)
```

Listing 5.1. Score aggregation. Per-metric contributions are computed as 100 x weight x score before this function is called.

```
# grader/models.py: fixed evaluator configuration
class LLMSettings(StrictModel):
    model: str = "qwen2.5-coder:7b"
    temperature: Literal[0] = 0
    seed: int = 42
```

Listing 5.2. Configuration excerpt; additional context/output limits and validation fields are present in the source.

5.4 Compiler and sanitizer integration

The sanity build uses GCC with optimization and C11 warning flags. A separate strict build checks diagnostic hygiene. The memory-safety analyzer produces an instrumented binary with address/undefined-behavior sanitizers, disables recovery after supported findings, and evaluates the specified cases. Compiler and sanitizer results remain tool evidence, not LLM-generated statements.

```
# Representative sanity-build flags; executed inside Docker.
gcc -O2 -std=c11 -Wall -Wextra -Wpedantic main.c -o program

# Instrumentation used by the memory-safety specialist includes:
-fsanitize=address,undefined -fno-omit-frame-pointer
-fno-sanitize-recover=all
```

5.5 Failure policy in code

A failed compilation or smoke run is a property of the submitted program and leads to the documented skipped execution metrics. A malformed model response, unavailable service or analyzer exception is a property of the evaluation process and produces an incomplete report. The two conditions must not be represented by the same unexplained zero.

5.6 React frontend and FastAPI backend

The React interface provides a source editor, file upload, visible rubric, JSON rule editor, report history and expandable metric feedback. The backend exposes real queued/running/completed/error states; the interface does not simulate progress or fabricate grades. A production build is served by the same Python process as the API [5,6].

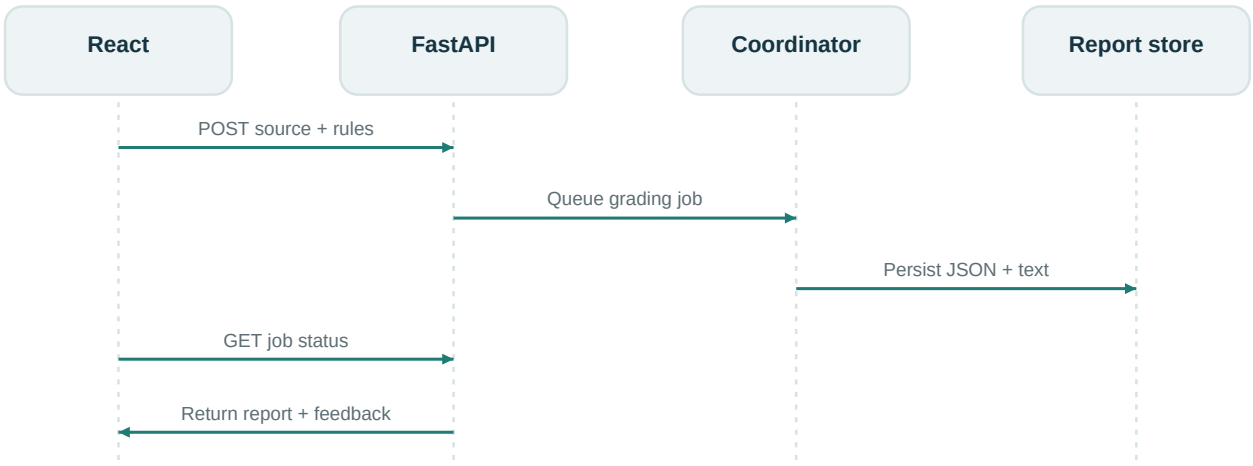


Figure 5.1. Frontend/API sequence. The full assignment report is returned inside the completed job response.

Endpoint	Purpose
GET /api/health	Check platform, Docker image and configured local model readiness.
GET /api/rules	Return the default self-contained rule configuration.
POST /api/evaluations	Submit filename, source and rules; receive a job identifier.
GET /api/evaluations/{id}	Poll actual state and obtain report/text when finished.
GET /api/reports	List recent saved report summaries.
GET /api/reports/{id}/{file}	Retrieve a saved JSON or text report.

5.7 Reports and interface boundaries

Each report carries the submission name, rules fingerprint, grade, first-pass status, metric contributions, evidence, flags and tool versions. It also retains runtime identities and the effective rubric. This is a local instructor interface: rules are editable and test inputs are visible. An authenticated public student portal would need separate access controls and protected-test views.

5.8 Team allocation and four-week plan

TEAM EVEREST consists of the four members listed below. Module assignments are proposed for team coordination and should be confirmed by the members; they are not asserted as verified individual contribution histories.

Member / roll number	Proposed ownership
Kanishk Mishra **2611CS20**	Rules, shared contracts, coordinator, scoring and verdict-cache review.
Roshan Jha **2611CS23**	Docker worker, compilation, I/O testing, edge oracle and sanitizer acceptance.
Mantosh Ravi **2611CS09**	Ollama transport, evaluator prompts and four semantic specialist rubrics.
Shivanshu Raj **2611CS26**	React frontend, FastAPI integration, report workflows and UI/API verification.

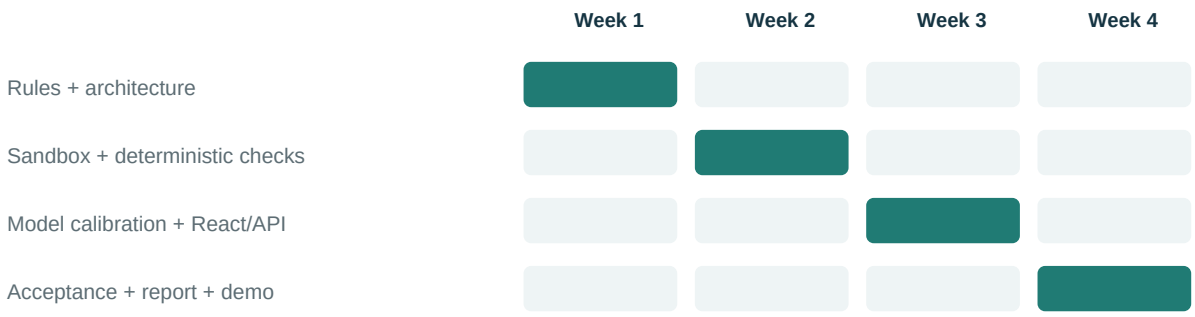


Figure 5.2. Proposed four-week schedule aligned with the assignment. This is a plan, not a measured effort or completion chart.

Week 1 establishes the rule contract and architecture. Week 2 validates the sandbox and deterministic tools. Week 3 calibrates local-model reviews and integrates the interface. Week 4 collects real Ubuntu results, finalizes documentation and prepares the demonstration. Cross-review should cover both code behavior and the accuracy of reported evidence.

Implemented components and successful software-level checks are recorded in Chapter 6. Remaining work includes real runtime/model acceptance, calibration against instructor-scored examples, genuine sample report generation, team review, repository publication and the required demonstration video.

CHAPTER 6
TESTING AND RESULTS

6.1 Testing approach and verified results

Tests were divided into software-level checks, Docker integration checks and an end-to-end model/runtime check. The creation host was Windows; Docker and Ollama were not available there. Consequently, test doubles were used only where explicitly required to verify orchestration and API behavior. No test-double score is presented as a real grading result [2].



Figure 6.1. Recorded test inventory: 43 discovered, 34 passed and nine skipped. These counts measure software checks, not student success rates or model accuracy.

Area	Observed verification
Rules / scoring	Weight totals, unique metric names, score bounds, fixed temperature and weighted contributions.
Orchestration	Compile/smoke dispatch, timeout flags, error handling, failing-input evidence and cache replay/invalidation.
API contracts	Job lifecycle, invalid weights, safe filenames, request cap, origin checks and visible service errors.
Build / browser	React production build; HTTP 200; rendered source/rubric; rule validation; navigation and backend platform error.

6.2 Pending target-environment tests

Eight skipped Docker tests cover compilation failure, correct boundary output, timeout, crash, output flood, memory cap, UBSan overflow and supervisor isolation. One skipped end-to-end test checks actual Docker/Ollama grading and cached replay. These checks must be run on Ubuntu before claiming complete execution safety or real model performance.

6.3 Assignment sample and defined arithmetic behavior

```
#include <stdio.h>
int main(void) {
    int a, b;
    scanf("%d %d", &a, &b);
    printf("%d\n", a + b);
    return 0;
}
```

The sample above represents the assignment's addition program. Signed integer overflow is undefined in C; it is not a portable promise of wraparound. The implemented rules instead define signed int32 inputs and require the mathematical sum. The corrected sample uses `long long` for input and arithmetic within that domain.

Input	Expected mathematical output	Case
7 5	12	Ordinary addition
-7 -5	-12	Negative operands
2147483647 1	2147483648	Upper boundary
-2147483648 -1	-2147483649	Lower boundary

Table 6.1. Oracle-defined expected outputs. These are not claimed execution results from the target system.

6.4 Illustrative scores versus measured results

The assignment's displayed contributions add to 79/100, although the example states 89/100. It also describes 8/10 passing cases while showing a 0.90 score. These values are not used as benchmark results. The implementation derives fractions and totals from actual analyzer outputs and rejects invalid configurations [1].

6.5 Reproducibility acceptance procedure

On Ubuntu, run a correct submission, the original sample, a compilation failure and a timeout fixture. Retain JSON/text evidence and model/image identities. Repeat unchanged requests to verify cached replay. Separately compare fresh recomputations under a fixed environment and investigate any score or timeout variation. The sample driver creates real reports after services are available; the report intentionally contains no invented student grades.

CHAPTER 7
USEFULNESS AND LIMITATIONS

7.1 Practical uses

University programming assignments

For a class of 300 addition submissions, the same rubric and numeric-boundary tests can be applied consistently. An instructor can inspect compilation failures and disputed deductions, while students receive a concrete failing input and expected sum instead of an unexplained mark.

Recruitment coding-test assistance

Reviewers can organize 50 C exercises using correctness and per-metric evidence, then focus on reasoning and design tradeoffs. Automated grading should support a human assessment process rather than become the sole basis for a consequential selection decision.

Independent programming practice

A learner can submit a nonterminating solution, see the timeout indication, fix the loop and rerun the same rules. Counterexamples are especially useful when ordinary inputs pass but negative values, zero or numeric extremes expose a mistake.

Code review and similarity triage

A strong match against a saved tutorial, peer solution or known generated example identifies a specific pair for inspection. This reduces search effort for an instructor without turning a statistical similarity signal into an accusation.

7.2 Conditions for useful feedback

The rubric must match the problem's learning objectives. Test cases should cover relevant boundaries, and semantic deductions should identify concrete source locations and improvements. Instructors should calibrate qualitative scoring on manually reviewed examples and retain a way to review unusual solutions or challenge a deduction.

7.3 Concrete unreliable-verdict cases

Limitation	Example of an unreliable verdict	Mitigation / boundary
Finite test coverage	A program hard-codes known inputs, passes every supplied/generated case and fails an unseen valid pair.	Add hidden/property-based cases and trusted oracles; finite tests do not prove correctness.
Model reasoning error	An input-dependent loop hidden in a macro is incorrectly classified as O(1); a conventional name receives an unfair deduction.	Calibrate rubrics, inspect source findings and preserve uncertainty.
Authorship ambiguity	Two independent five-line addition solutions exceed a similarity threshold; renamed copied code evades it.	Use local matches only for manual triage; do not claim proof of copying or AI authorship.
Execution / inference variability	A near-timeout program changes outcome under load; ASan overhead or a different model runtime changes the verdict.	Fix the environment, use an explicit sanitizer budget and distinguish replay from fresh recomputation.

Table 7.1. Four concrete cases where a confident automated verdict could be wrong or unreliable.

7.4 Security and human-review boundary

Docker provides a useful local restriction boundary but shares the host kernel. A hostile public judge needs stronger isolation, authentication, queue isolation, quotas and operational hardening. A container-wide resource failure can still interrupt the supervisor and must be reported as an incomplete evaluation.

Prompt boundaries similarly reduce, but do not eliminate, susceptibility to instructions embedded in student comments. The model cannot call tools through this interface, yet its qualitative judgments still require calibration. A reproducible cached verdict can remain wrong; auditability is not proof of correctness.

CHAPTER 8
CONCLUSION AND FUTURE WORK

8.1 Conclusion

AutoGrader integrates external rules, restricted execution, independent analyzers, local model review and weighted reporting into one modular application. The React frontend and FastAPI backend make source submission, rule inspection and feedback accessible while keeping policy in the Python evaluation layer. Direct file processing is sufficient for the bounded project scope and avoids unnecessary retrieval infrastructure.

The codebase includes nine analyzers, sample submissions, automated tests, reproducibility metadata and Ubuntu setup scripts. The recorded evidence establishes software-level integration through 34 passing tests, a successful React build and selected browser/API checks. It does not yet establish real Docker/Ollama grading performance on Ubuntu. That boundary must remain explicit in the final project demonstration.

8.2 Future work

1. Complete target-host acceptance. Run the nine skipped checks, collect genuine sample reports and review timeout, sanitizer and model-failure behavior.

2. Calibrate semantic scoring. Compare specialist deductions with instructor-scored examples, revise task-specific anchors and quantify disagreement before consequential use.

3. Extend test generation. Add trusted oracles and property-based cases for new C problems; support multi-file submissions only after dependency and isolation policies are defined.

4. Strengthen isolation and access. Add a dedicated VM/microVM execution layer and separate instructor/student views before public deployment.

5. Improve analysis evidence. Consider reviewed static-analysis tools and controlled measurements while avoiding claims that timing alone proves asymptotic complexity.

8.3 Final submission checklist

The team should confirm its responsibility allocation, review the code and report, publish the actual repository, archive runtime identities and rules, attach real outputs, and record the assignment demonstration. No repository URL, execution grade or individual contribution history should be filled with an invented value.

APPENDIX A
EXAMPLE RULE CONFIGURATION

The following is a valid, reduced two-metric configuration for demonstrating the rule interface. Its weights sum to 1.00. The complete implemented nine-metric configuration, including all semantic rubrics, is provided in `rules/add.json`. This reduced example is not the default grading policy.

```
{
  "problem": "add_two_integers",
  "specification": "Read two int32 values; print the sum and newline.",
  "time_limit_s": 1,
  "memory_limit_mb": 64,
  "comparison": "exact",
  "tests": [
    {"name": "sample", "input": "7 5\n", "expected": "12\n"},
    {"name": "boundary", "input": "2147483647 1\n",
     "expected": "2147483648\n"}
  ],
  "metrics": [
    {"name": "correctness", "analyzer": "io_tests",
     "weight": 0.80, "params": {}},
    {"name": "syntax", "analyzer": "compiler_warnings",
     "weight": 0.20, "params": {}}
  ],
  "llm": {
    "model": "qwen2.5-coder:7b",
    "temperature": 0,
    "seed": 42
  }
}
```

The evaluator still records the configured local model identity, even when the selected reduced metrics are deterministic.

A.1 Rule-editing guidance

Change tests, weights, limits and rubric anchors in the JSON. Use inline tests in the web editor. For CLI-only external pairs, paths must remain below the rule directory. Add subjective criteria with the `llm_rubric` analyzer; implement a new class for a genuinely new tool capability. Arbitrary compiler flags and executable imports are not accepted from rules.

APPENDIX B
FEEDBACK AND JSON REPORT CONTRACT

The backend preserves the assignment's main JSON fields and adds audit metadata. The table below defines the report shape without inventing a measured score or pretending that an unexecuted sample has been graded.

Field	Meaning
submission	Original source filename.
rules_hash	SHA-256 fingerprint of the effective rule configuration.
grade	Weighted score out of 100, or null when the evaluation is incomplete.
first_pass	Compilation, smoke-run and suspected-infinite-loop status with evidence.
metrics	One result per configured metric: score, status, weight, contribution, evidence and feedback.
flags	Compilation/runtime warnings or a similarity-review flag.
tool_versions	Recorded GCC and Ollama information when available.
usefulness_limitations	Concrete practical uses and limitations attached to the report.
Audit fields	evaluation_id, source_hash, runtime_identity, llm_settings, rules_snapshot and cache_hit.

B.1 Failure evidence

An output mismatch records the test name, input, expected answer, observed output, process status, return code and standard-error diagnostics. A source-review finding records a real source line and an actionable explanation. A similarity result names a reference and its measured token similarity while retaining the authorship uncertainty.

B.2 Report storage and transport

```
data/reports/<evaluation-id>/add_original.json
data/reports/<evaluation-id>/add_original.txt

POST /api/evaluations
GET  /api/evaluations/<job-id>
```

The HTTP job wrapper contains real queued/running/completed/error state. When complete, it carries the structured `report` and the readable `text`. A completed request can still contain an incomplete grade if one of the analyzers could not produce valid evidence.

APPENDIX C
IMPLEMENTATION EVIDENCE

C.1 Codebase, environment and recorded checks

Item	Recorded status
Source package	Modular Python/React project delivered as autograder.zip; repository publication is pending.
Creation host / target	Windows creation and software-level verification; Ubuntu 24.04 target for grading.
Observed web tools	Vite 7.3.7; FastAPI 0.162.0; Uvicorn 0.54.0 in the preview/test environment.
Automated tests	43 discovered; 34 passed; nine skipped because Docker/Ollama were unavailable.
Production build / API	React assets built; local FastAPI page returned HTTP 200.
Browser checks	Source/rubric rendering, rules editing/validation, view navigation and backend error display.
Actual grading results	Not yet collected from Ubuntu. No real student grade is claimed in this report.

C.2 Ubuntu setup and acceptance commands

```
# Prerequisites: Python 3.10+, Node.js 22.12+, Docker, Ollama.
bash scripts/setup.sh
ollama pull qwen2.5-coder:7b
docker build -t autograder-sandbox:1 sandbox/
.venv/bin/python app.py doctor
bash scripts/start.sh
# Open http://127.0.0.1:7860

# Run real integration checks on the target host:
RUN_DOCKER_TESTS=1 RUN_E2E_TESTS=1 \
  .venv/bin/python -m unittest discover -s tests -v
.venv/bin/python run_samples.py
```

Preserve the installed dependency lock, image identity, model digest, test rules and verdict cache for the demonstration. The test log and genuine report files should accompany the final repository once target-host acceptance is complete.

C.3 Actual React interface preview

Figure C.1 shows the locally rendered interface from the implemented React application. The visible setup message is intentional: this host can display the frontend and API, but it cannot perform real Linux grading without the required runtime services. The screenshot is evidence of interface rendering, not evidence of a successful C evaluation.

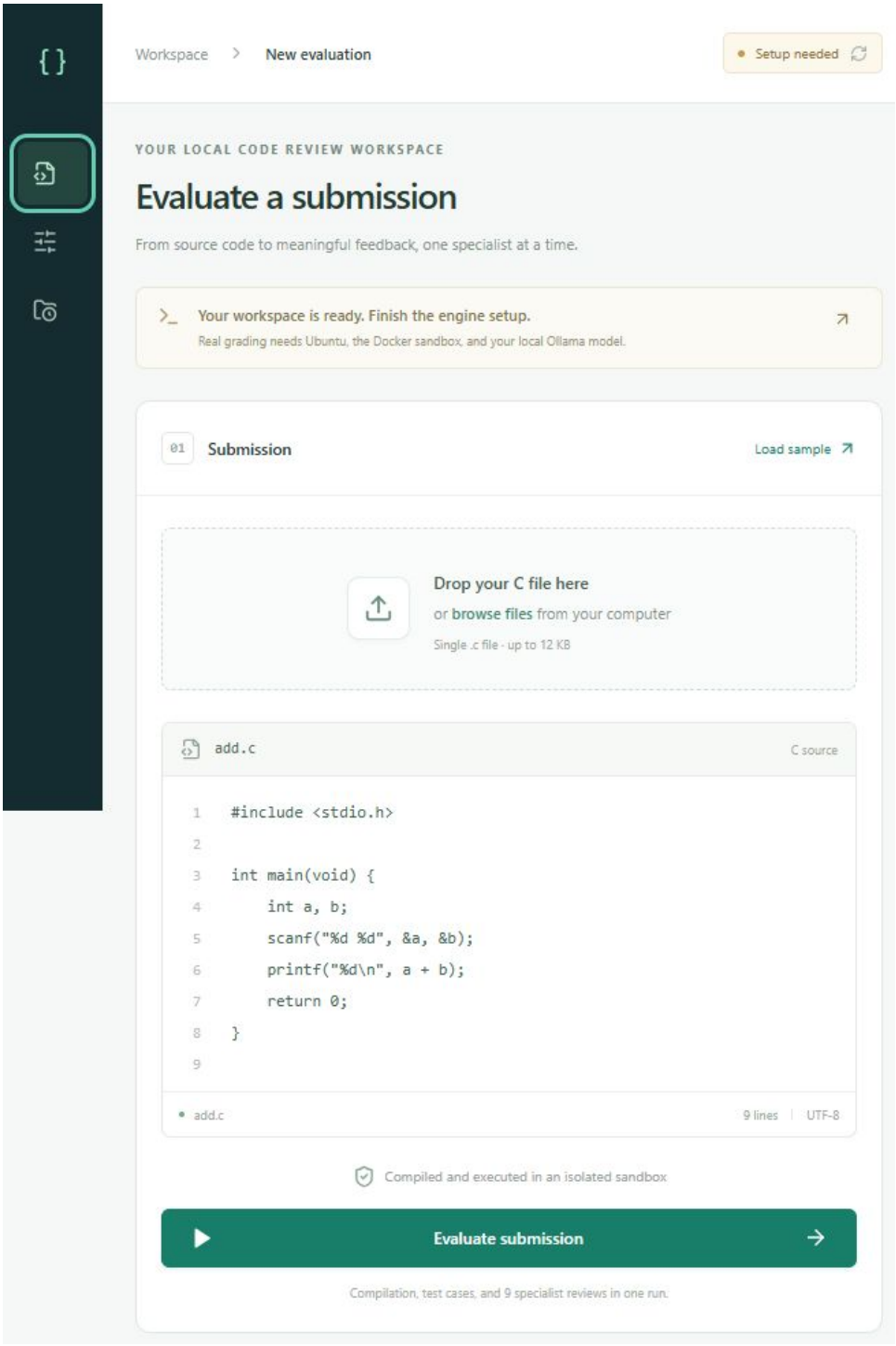


Figure C.1. Actual React source-upload/editor view, cropped from the local interface capture. No score is displayed because real grading was not performed on this host.

REFERENCES

[1] *AutoGrader: A Rule-Driven Evaluator for C Programs.* Supplied Agentic-AI-assignment.pdf, pp. 1-9. Assignment requirements, example rubric and submission expectations.

[2] AutoGrader project implementation and validation artifacts. `docs/REPORT.md`, `docs/VALIDATION.md`, `rules/add.json` and source/tests. Local project record, October 2026.

[3] Ollama documentation. *Generate a chat message*; *Structured outputs*; *List models.*
https://docs.ollama.com/api/chat
https://docs.ollama.com/capabilities/structured-outputs
https://docs.ollama.com/api/tags

[4] Docker documentation. *Running containers*; *Docker Engine security*.
https://docs.docker.com/engine/containers/run/
https://docs.docker.com/engine/security/

[5] FastAPI documentation. API routing, validation, testing and static-file serving.
https://fastapi.tiangolo.com/

[6] React documentation and Vite guide. Component-based frontend development and asset building.
https://react.dev/
https://vite.dev/guide/

[7] *AgenticReport2.pdf.* User-supplied format reference. Used for the formal title-page structure, institutional/supervisor details, chapter organization and appendices. The institute emblem is retained from that reference.